



Aprendizagem Automática 2023/2024

K-nearest neighbours for regression and classification
Preprocessing data: scaling and imputation

André Falcão

(aofalcao@ciencias.ulisboa.pt)

1

Summary

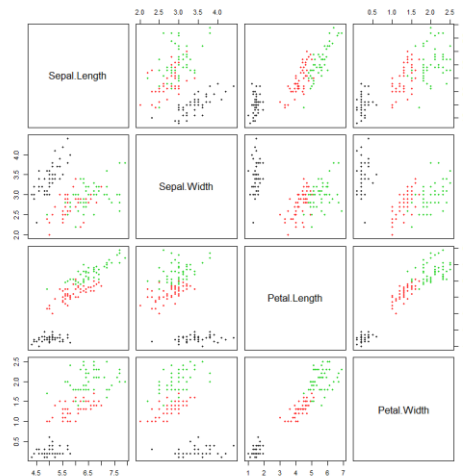
- Vector vs Metric spaces
- Distance metrics
- K-nearest neighbours
 - Classification
 - Regression
- Approaches to data scaling
- Missing values imputation

2

Handling data and making models

- Looking at data

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
4.9	3.0	1.4	0.2	setosa
5.8	2.7	3.9	1.2	versicolor
6.1	2.8	4.7	1.2	versicolor
5.2	4.1	1.5	0.1	setosa
7.4	2.8	6.1	1.9	virginica
5.2	3.5	1.5	0.2	setosa



3

Problems with vectors

- Not all data can be represented as a vector. Eg.
 - Strings,
 - Proteins
 - images
- But many times we can compute **similarities** between instances



4

Problems with this approach

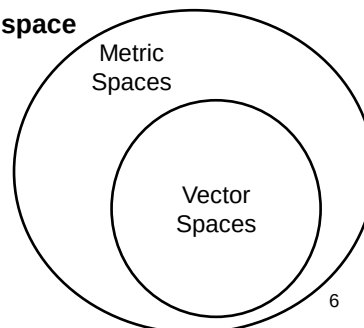
- Not all data can be represented as a vector
- But many times we can compute **similarities** between instances



5

Vector spaces vs metric spaces

- A vector space is a feature space, where each instance can be described as a set of measured features
- A metric space is a space where instances actually can be defined relative to each other by way of a distance or a similarity function
 - A **vector space is always a metric space** because we can compute distances between vectors
 - Euclidean Distance
 - Minkowski
 - Manhattan
 - Jaccard....

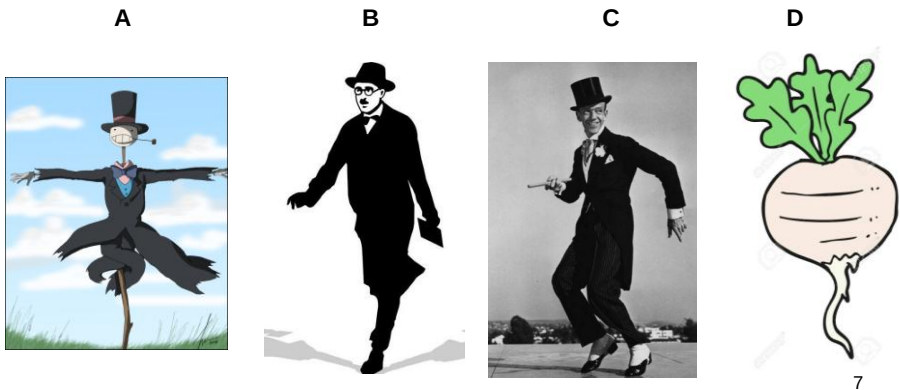


6

The question of similarity

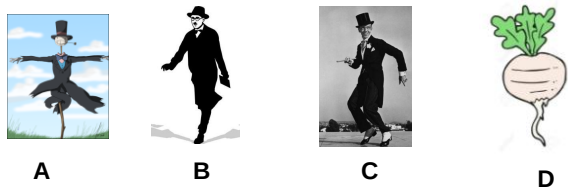
- How similar?

© 2020-2022 - Docentes MC - DI/FCUL



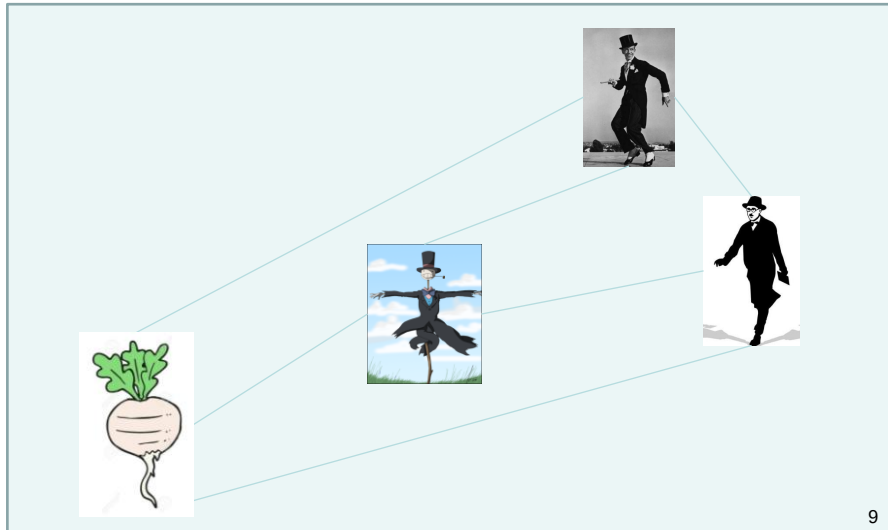
Similarity matrix

© 2020-2022 - Docentes MC - DI/FCUL



A	1			
B	0.5	1		
C	0.6	0.8	1	
D	0.6	0.05	0.05	1
	A	B	C	D

A 2D projection



Measuring distances

- Euclidean Distance

$$D(X, Y) = \sqrt{\sum_i (x_i - y_i)^2}$$

- Manhattan Distance

$$D(X, Y) = \sum_i |x_i - y_i|$$

- Minkowski distance

- A generalization
- For $p=1$ - Manhattan distance
- For $p=2$ - Euclidean distance
- For p very large ~ Chebyshev distance - largest absolute distance between features

$$D(X, Y) = (\sum_i |x_i - y_i|^p)^{\frac{1}{p}}$$

Distances for categorical attributes

- Most times categorical attributes are presented to models as 0/1 columns, indicative whether or not they have a given characteristic
 - Taxonomic trees
 - Presence of fragments or pieces
 - Words in strings
- There are many distances very common for those problems
 - Jaccard Similarity and Distance
 - Cosine Distance
 - Dice
 - Overlap
 - Longest common subsequence [not for all cases]

1	[0 1 1 0 0 1 0 1]
2	[1 0 1 0 1 1 0 0]

11

The Jaccard Similarity/Distance

- This very popular similarity metric is based on representing items as binary values
 - Presence or absence of a characteristic
 - Very, very common in many fields
 - Presence or absence of a word, an n-gram, a string of characters,...
- Jaccard Similarity is the ratio between coincident 1-bits and all existing 1-bits between 2 items

$$JaccardSim(X, Y) = \frac{X \cap Y}{X \cup Y}$$

$$JaccardDist(X, Y) = 1 - JaccardSim(X, Y)$$

- Example

1	[0 1 1 0 0 1 0 1]
2	[1 0 1 0 1 1 0 0]

- Intersect (D1,D2) = 2
- Union(D1,D2) = 6
- JaccardSim = 2/6 = 0.3333
- Jaccard Distance = 1 - JaccardSim = 0.6667

12

Cosine similarity

- The cosine similarity is very common where there are many sparse columns. E.g., text processing
- Focus on the amount of common bits, divided by the product of the norms.

$$D(X, Y) = \frac{X \cdot Y}{\|X\| \cdot \|Y\|}$$

- Similar to Jaccard, only the 1 bits are counted, thus focusing on existence of characteristics rather than common absence

```
1 [0 1 1 0 0 1 0 1]
2 [1 0 1 0 1 1 0 0]
```

- CosineSim = $2/(\sqrt{4} \cdot \sqrt{4}) = 2/4 = 0.50$
- CosineDist = $1 - 0.50 = 0.50$

13

Other common metrics

- Other distance metrics

– Dice-Sorenson

$$DiceSim(X, Y) = \frac{2|X \cap Y|}{|X| + |Y|}$$

– Overlap

$$OverlapSim(X, Y) = \frac{|X \cap Y|}{\min(|X|, |Y|)}$$

- Longest common subsequence – only applicable for sequences like strings (text, proteins, DNA,...) when we want to count the amount of common sequential elements

		A	B	C	D	A
	0	0	0	0	0	0
A	0	1	1	1	1	1
C	0	1	1	2	2	2
B	0	1	2	2	2	2
D	0	1	2	2	3	3
E	0	1	2	2	3	3
A	0	1	2	2	3	4

LCS = "ACDA"

4

<https://algorithms.wtf/dynamic-programming-longest-common-subsequence/>

© 2020-2022 - Docentes MC – DI/FCUL

-
- | | | | | | | | |
|--|--|--|--|--|--|--|--|
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
- h y u n d a i
- h o n d a
- h o n d a
- h y u n d a i

The flowchart illustrates the dynamic programming algorithm for protein alignment. It begins with two protein structures, A and B, which are compared based on their vector environments (selected χ residue pairs, having similar accessibility and physical angles). This comparison leads to the generation of a 'Vector view for protein A' and a 'Vector view for protein B'. These views are then used to create a 'Residue level score matrix'. A 'Summary score matrix' is then generated from the residue level score matrix. The summary score matrix is used to generate a path for each β comparison with a dynamic algorithm. If the score is greater than a threshold, the path is added to the summary score matrix. Finally, the dynamic algorithm is applied to the summary score matrix to generate the final alignment.

K-Nearest Neighbour

-

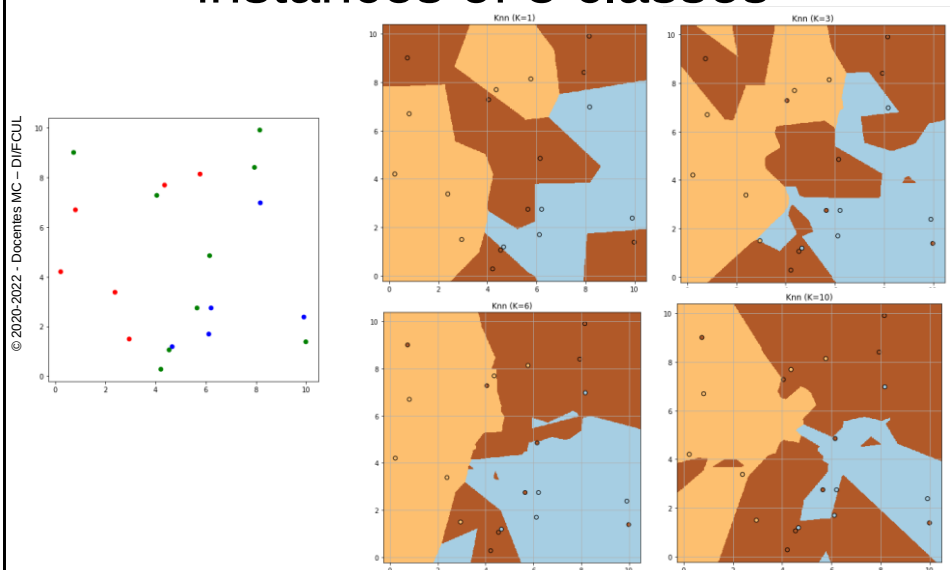
16

KNN for Classification

- KNN for classification can lead to ties, even if K is odd.
 - E.g., Classification with K=5 and 3 classes:
 - $NN=\{“A”, “B”, “A”, “B”, “C”\}$ What should the final class be?
 - Ties will produce an output that many times is dependent on the data order
 - Using the shortest distance to new instances may help, but not always
- Typically increasing the number of neighbours will increase the homogeneity of the prediction region, but depends a lot on the data topology
 - K=1 – typically high variance
 - K large – typically high bias

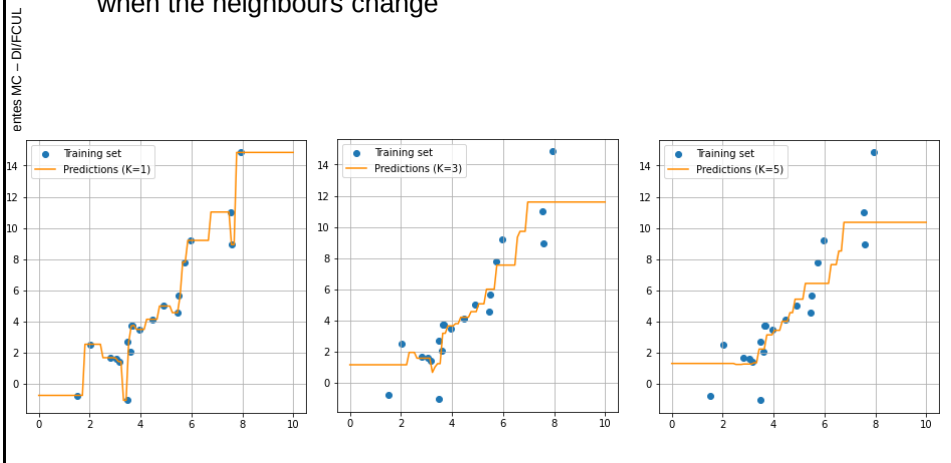
17

Example with 20 random instances of 3 classes



KNN for regression

- KNN for regression typically uses the mean of all the neighbours for making predictions, which can lead to step increases, changing only when the neighbours change



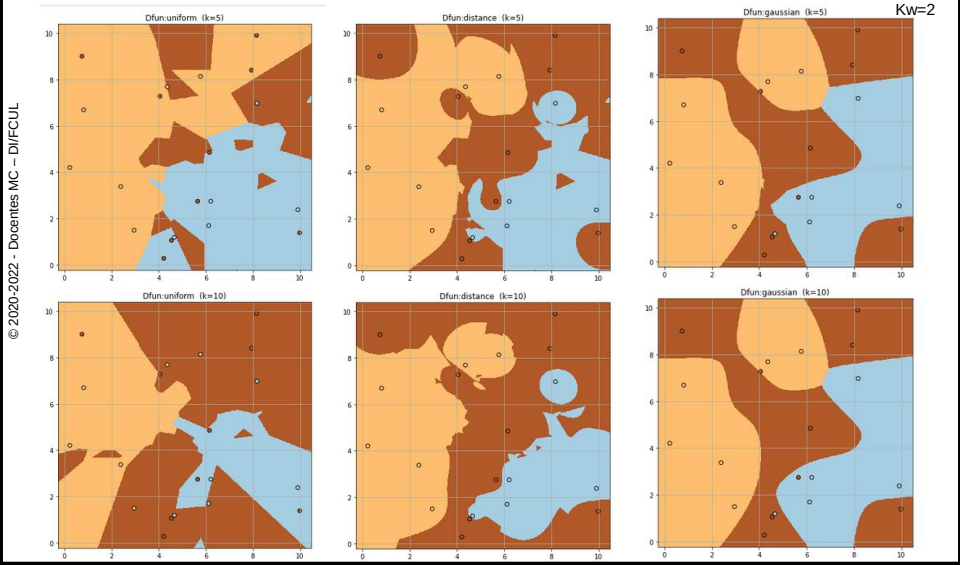
KNN distance weighting

- Sometimes it may be useful to apply weights to each prediction according to the distance each new instance may be from each instance of the training set
- There are two common strategies
 - Inverse distance weight each prediction according to the inverse of the distance, meaning that farther away training instances will have a smaller importance
 - Gaussian kernels – have a similar effect but are smoother. Furthermore require a kernel weight (kw) parameter that needs to be estimated separately

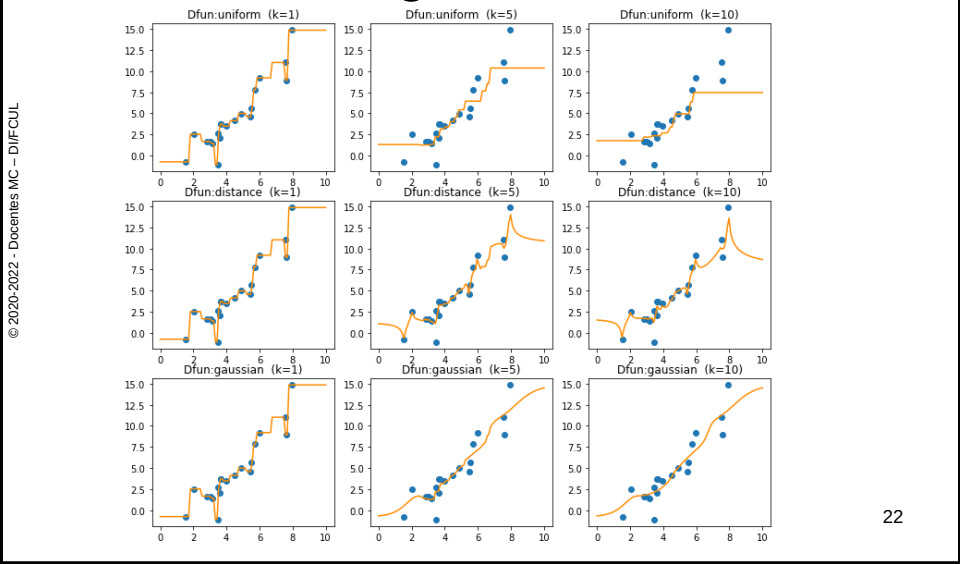
$$w_i = \frac{1}{d_i}$$

$$w_i = e^{-\frac{d_i^2}{kw}}$$

Distance weighting functions on Classification



Distance weighting functions on regression



KNN optimizations

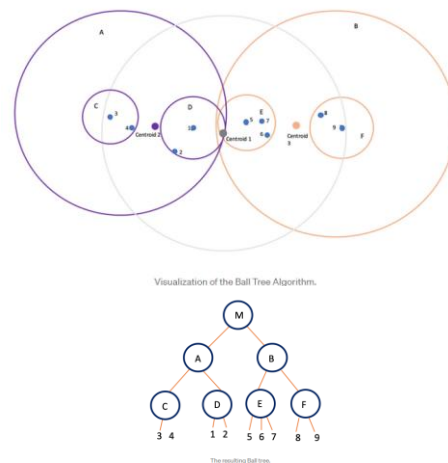
- KNN is intrinsically slower than most supervised learning
 - Time complexity: $O(N.M)$ – Grows linearly with the size of the data set
 - Sometimes may be impractical for very large data sets
- Yet very common and easy to parallelize
 - embarrassingly parallel problem
 - For moderate model use with small-medium training sets
 - Use parallel jobs (threads or cores)
 - For heavy use with large training sets
 - Split data in different servers
- Strategies for use in large and complex datasets
 - Nearest neighbour algorithms
 - Ball tree
 - K-Dimensional tree (KDtree)
 - No NN algorithm is perfect and may miss some values, but most of the times it is sufficiently good

23

© 2020-2022 - Docentes MC - DIFCUL

Ball Tree Search

- Searching for neighbours is a complex procedure involving searching the full dataset
 - The only algorithm that is perfect is “brute force” approach
- The ball tree searches for instances in a metric space with centroids and appropriate radius that overlap.
 - The search procedure goes down the intersecting “balls” (hyperspheres)
 - The purpose is to trim down and ignore too far away points
- KD Tree is a variation of binary space partitions with orthogonal space splits



<https://towardsdatascience.com/tree-algorithms-explained-ball-tree-algorithm-vs-kd-tree-vs-brute-force-9746debcd940>

24

© 2020-2022 - Docentes MC - DIFCUL

Is K nearest Neighbours

- Simple?
 - **Not really** – Even though the principles are simple, the problem of inference grows linearly with the size of the dataset. The model is the dataset
- Stable?
 - **YES (kind of)** – The result is most of the times the same, Yet in classification when there are ties, the result may depend on the training set order
- Fast to learn?
 - **YES** – No training. The model is the data [eventually some processing for space partitioning]
- Fast to make predictions?
 - **NO** – Typically for large datasets it is a slow algorithm
- Updateable?
 - **YES** – more data is trivially added to the model without any problem or refitting. Yet, eventually it may be required to make a new data partition for efficient searching.

© 2020-2022 - Docentes MC - DII/FCUL

25

Scaling data

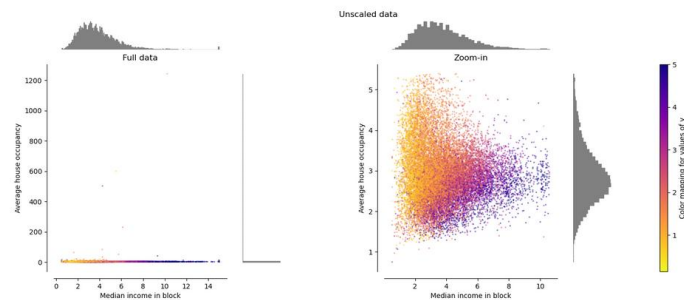
- Why scale data?
 - Many modelling approaches (especially those based on distances!) require some uniformity in the independent variables
 - Some datasets have outliers that may totally disrupt model results
 - Occasionally, the raw data only makes sense if properly scaled
- How to scale data?
 - **The scaling parameters are model parameters!**
 - They must be fit only with the training data
 - This is a problem in many test-validate approaches (e.g. Kfold cross validation)
 - Each new data to be fit by the model needs to be scaled before entering the model

© 2020-2022 - Docentes MC - DII/FCUL

26

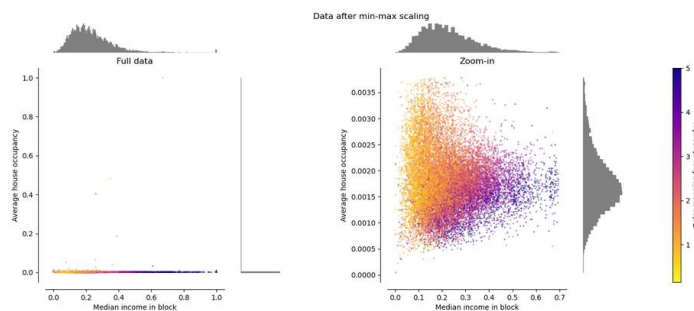
Common data scaling methods

- Standardization
- Range scaling
- Power transform
- Unit norm scaling
- Custom scaling



Range Scaling

- Range scaling is useful when models require a specific range of values
- Common ranges are $[0,1]$ or $[-1, 1]$
- Outliers will be an issue and may compress data



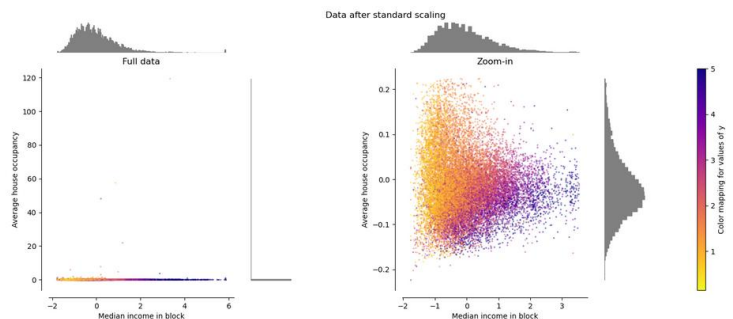
28

https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html#sphx-glr-auto-examples-preprocessing-plot-all-scaling-py

Standardization Scaling

- This is one of the most popular scaling techniques. Subtracts the mean and divides by the standard deviation
- Makes all variables directly comparable
- Does not handle outliers

$$X_i = \frac{X - \mu}{\sigma}$$



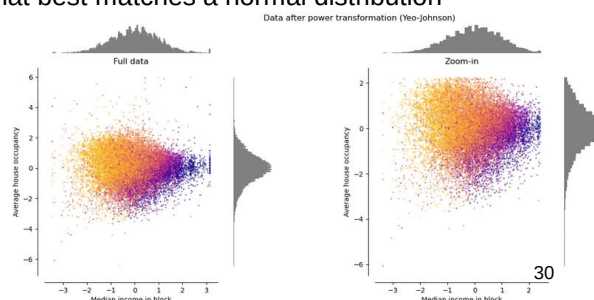
Power Transform

- This is a way of handling outliers by transforming data into a normal distribution via a power transformation. Eg.

- Box-Cox –for strictly positive features only
- **Yeo-Johnson**

$$y_i^{(\lambda)} = \begin{cases} ((y_i + 1)^\lambda - 1)/\lambda & \text{if } \lambda \neq 0, y \geq 0 \\ \log(y_i + 1) & \text{if } \lambda = 0, y \geq 0 \\ -((-y_i + 1)^{(2-\lambda)} - 1)/(2 - \lambda) & \text{if } \lambda \neq 2, y < 0 \\ -\log(-y_i + 1) & \text{if } \lambda = 2, y < 0 \end{cases}$$

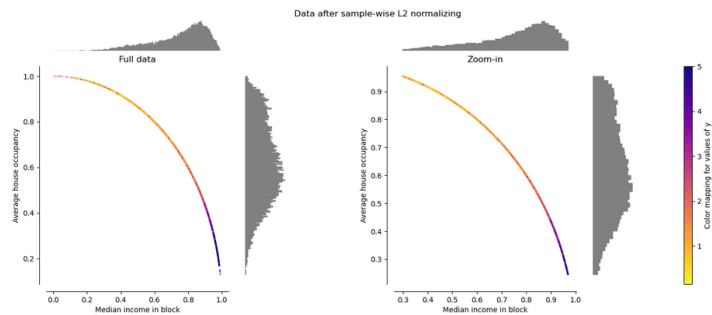
- The algorithm searches many values of lambda and selects for each feature the one that best matches a normal distribution
- Handles outliers, compressing them



Unit norm scaling

- Disregards separate features and makes certain each instance has its norm= 1
- Handles outliers (but these may dominate some samples)
- Very useful in text processing for computing similarities between texts

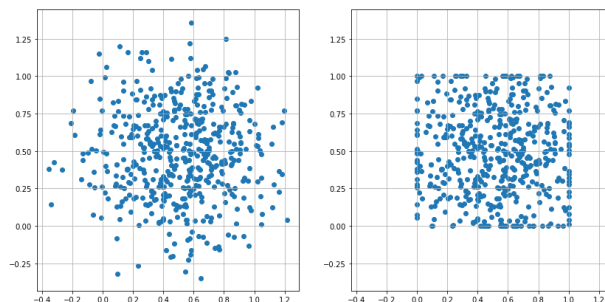
© 2020-2022 - Docentes MC - D/FCUL



Custom scaling

- Some types of scaling may be simple mathematical transforms
 - $\log()$, $\sqrt{}$ are many times used
 - This is done when we apply domain knowledge to each variable
- A particular case is Linear clamping
 - Sometimes it may be desirable to simply remove the extremes, or consider them grouped together by not allowing them to be larger or smaller than a specific value
 - Here $[0,1]$ clamping
 - Loss of information
 - Can handle outliers but not always the best approach

© 2020-2022 - Docentes MC - D/FCUL



Handling missing values

- What to do when data is missing?
- 4 basic approaches
 - **Do nothing** – some models do not intrinsically require all the variable data (e.g. Naive Bayes)
 - **Eliminate Rows**
 - If not many instances are missing feature data
 - **Eliminate features** (columns)
 - If some columns contain all the missing values and do not appear important for inference
 - Combinations of the above (remove rows and columns)
 - **Impute data**
 - For sparse missing data ranging several rows and features

© 2020-2022 - Docentes MC - DII/FCUL

33

Basis of data imputation

- **Essentially is a modelling process**
 - As any modelling process, must always be made with the training set
 - The model for imputation is part of the global model and must always be used before the main inference model
- We want to put on the missing cells some values that will not disrupt the final model
 - Identify the **maximum likelihood** value to fill in the missing values so that the overall dataset properties do not change
- Two Approaches:
 - Univariate imputation – Looking only for each column independently, and replace missing values with a reasonable estimate
 - Multivariate estimate – Looking at each missing value as a unique issue and fitting an adequate multivariate model to all the data to fill in the missing information

© 2020-2022 - Docentes MC - DII/FCUL

34

Univariate imputation

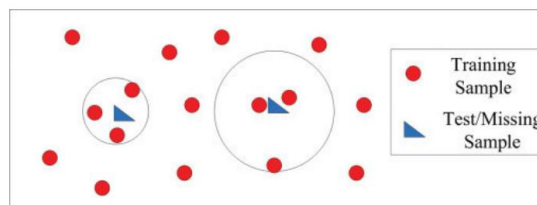
- Replace each missing value with a common statistic or assumption
 - Mean
 - Median
 - Mode (most frequent value)
 - Constant fixed value
- Very simple approach but it works quite well if not too many missing values
- The impact on a inference model should be checked:
 - Make an inference model using median imputation (as an example)
 - Make an inference model using a constant value for all missing values (like 0 or 999) – can be an outrageous value!
 - Compare the impact of the actual imputation with the constant value in the model statistics

35

© 2020-2022 - Docentes MC - DII/FCUL

Multivariate Imputation

- KNN Imputation
 - Essentially uses the closest instances with common (non missing) features to infer the missing values
 - Is a multivariate approach that uses known values of similar data for estimation
 - Can be slow and may compromise the speedy use of a model if it is common to have several missing values
- Other modeling approaches have been used with success (e.g. Naive Bayes)



https://www.researchgate.net/figure/Regression-and-missing-value-imputation-using-the-kNN-method-with-a-fixed-k-value-that_fig5_312507655

36

© 2020-2022 - Docentes MC - DII/FCUL

Expectation maximization

- EM is a general greedy method for estimating statistics and data properties when not all information is available
- Steps
 1. Compute actual population statistics for all variables
 2. Impute the data using a **maximum likelihood** estimate (Expectation)
 3. With the new dataset estimation recalculate the new parameter estimates (Maximization)
- This procedure is repeated several times until convergence
- Requires knowledge of the actual data distribution for each variable

<https://towardsdatascience.com/handling-missing-data-like-a-pro-part-3-model-based-multiple-imputation-methods-bdfe85f93087>

37

Closing remarks on data preprocessing

- Data should be visualized and inspected before fitting a model
- Different features may require different types of data scaling and imputation.
 - Be wary of “**one size fits all**” approach
- Typically do data scaling BEFORE imputation
 - Scaling is (most commonly) a univariate process
 - Imputing data before scaling may change the overall properties of each feature and disrupt its contribution to the overall model
 - There are obviously exceptions (e.g. Unit Norm scaling does not allow missing values)
- Keep the data processing pipeline very well documented and follow it strictly for model use

38

Elements of study

- Hastie et al, 2009 – Chapter 2 (2.3) and Chapter 13
 - For EM algorithm Chapter 8 (8.5)
- Flach, 2012 – chapter 8 (8.1. to 8.3)

© 2020-2022 - Docentes MC - DI/FCUL

Next class

- Kernel based methods
 - Support Vector Machines

© 2020-2022 - Docentes MC - DI/FCUL